

MCP Courier

Reporte del proyecto

Humberto Alexander de la Cruz — Carné 23735
Universidad del Valle de Guatemala

01/09/2026

Índice

1. Especificación de los servidores MCP	2
1.1. Transportes soportados	2
1.2. Herramientas expuestas por <code>server-courier</code>	2
1.3. Servidores MCP oficiales utilizados	3
2. Análisis de la comunicación con Wireshark	4
2.1. Configuración de la captura	4
2.2. Clasificación de los mensajes JSON-RPC	4
2.2.1. Mensajes de sincronización	4
2.2.2. Mensajes de solicitud	5
2.2.3. Mensajes de respuesta	7
2.3. Análisis por capas	8
2.4. Resumen de la captura	9
2.5. Observaciones	9
3. Conclusiones y comentarios sobre el proyecto	10
3.1. Dificultades encontradas y cómo se resolvieron	10
3.2. Lecciones aprendidas	10
3.3. Comentario general	11

1 Especificación de los servidores MCP

Este proyecto implementa el protocolo MCP (*Model Context Protocol*) de forma manual: todo el intercambio de mensajes JSON-RPC 2.0 — framing, *handshake* de `initialize`, `tools/list` y `tools/call` — está codificado desde cero, sin utilizar ningún SDK de MCP. La versión de protocolo declarada por el servidor propio es 2025-11-25.

1.1 Transportes soportados

El servidor propio (`server-courier`) implementa dos transportes intercambiables sobre la misma lógica de negocio:

`stdio (local)`

Framing por línea: cada mensaje JSON-RPC ocupa una línea completa terminada en `\n`, escrita al `stdout` del proceso hijo (o leída de su `stdin`). Es el modo usado cuando el anfitrión (`client-host`) ejecuta el servidor como proceso hijo en la misma máquina.

`HTTP (remoto)`

Un único endpoint, `POST /mcp`, que recibe un mensaje JSON-RPC por cuerpo de petición. Puerto configurable mediante la variable de entorno `MCP_HTTP_PORT` (desplegado en el puerto 8080 en la instancia EC2 usada para este proyecto). Es el modo usado para el despliegue remoto.

En el transporte HTTP, el servidor emite un identificador de sesión al recibir `initialize`, devuelto en el encabezado de respuesta `Mcp-Session-Id`. El cliente debe reenviar ese mismo encabezado en toda petición posterior; una sesión desconocida o expirada resulta en `HTTP 404`. Las peticiones que corresponden a una *notificación* JSON-RPC (sin campo `id`, p. ej. `notifications/initialized`) se responden con `HTTP 202 Accepted` y cuerpo vacío, mientras que toda petición con `id` recibe `HTTP 200` con el resultado JSON-RPC correspondiente.

Los errores a nivel de protocolo utilizan los códigos estándar de JSON-RPC 2.0: `-32700` (JSON inválido), `-32600` (petición inválida), `-32601` (método no encontrado), `-32602` (parámetros inválidos) y `-32603` (error interno).

1.2 Herramientas expuestas por `server-courier`

Las seis herramientas se descubren mediante `tools/list` y se invocan mediante `tools/call`. Cada una define su `inputSchema` como JSON Schema.

`consultar_restricciones`

- **Parámetros:** `categoria` (*string*, requerido)
- **Retorna:** { `categoria`, `estado`: "permitido"|"restringido"|"prohibido", `requisitos` }

`calcular_dai`

- **Parámetros:** `valorDeclarado` (*number* ≥ 0 , requerido), `categoria` (*string*, requerido)
- **Retorna:** arancel aplicado, monto de DAI, IVA (12%) y total de impuestos

cotizar_importacion

- **Parámetros:** `valorDeclarado` ($number \geq 0$), `pesoLb` ($number \geq 0,1$), `categoria` (*string*) — los tres requeridos
- **Retorna:** desglose completo (flete, manejo, DAI, IVA, total a pagar) y tiempo estimado de entrega. Internamente verifica restricciones y calcula impuestos sin requerir llamadas adicionales del cliente.

registrar_paquete

- **Parámetros:** `cliente`, `descripcion` (*string*), `pesoLb` ($\geq 0,1$), `valorDeclarado` (≥ 0), `categoria` — todos requeridos
- **Retorna:** el registro creado, con número de guía generado (formato `GTC-XXXXXXX-XXXX`) y estado inicial `Recibido en bodega Miami`

rastrear

- **Parámetros:** `guia` (*string*, requerido)
- **Retorna:** estado actual e historial de eventos del paquete

listar_sucursales

- **Parámetros:** `departamento` (*string*, opcional)
- **Retorna:** listado de sucursales, filtrado por departamento si se especifica

Los datos de tarifario, aranceles y restricciones se leen de un archivo semilla (`data.json`, ilustrativo, no corresponde a cifras oficiales); los paquetes registrados se persisten en un archivo de estado independiente (`state.json`), regenerado a partir de la semilla en cada primer arranque.

1.3 Servidores MCP oficiales utilizados

Además del servidor propio, el anfitrión conecta dos servidores oficiales de `github.com/modelcontextprotocol/` ambos por `stdio` local:

- **Filesystem** (`@modelcontextprotocol/server-filesystem`): expone herramientas de lectura/escritura de archivos, restringidas por el propio servidor a un directorio permitido (`allowed directory`) indicado al arrancarlo.
- **Git** (`mcp-server-git`, oficial de Anthropic): expone herramientas como `git_status`, `git_add`, `git_commit` y `git_log`. A diferencia del servidor de Filesystem, este *no* restringe a un directorio fijo: cada llamada recibe un parámetro `repo_path` arbitrario, por lo que el aislamiento a un solo repositorio de trabajo se logra únicamente mediante instrucción al modelo de lenguaje (*system prompt*), no mediante una restricción impuesta por el servidor.

Como ambos son servidores de terceros (no desarrollados para este proyecto), no se detalla aquí la totalidad de sus parámetros; el anfitrión los descubre dinámicamente igual que al propio, mediante `tools/list`.

2 Análisis de la comunicación con Wireshark

Servidor remoto: 13.221.126.71:8080 (instancia EC2, Ubuntu 24.04). Cliente: 192.168.68.115 (anfitrión del chatbot, Node.js). Endpoint: POST /mcp sobre HTTP/1.1.

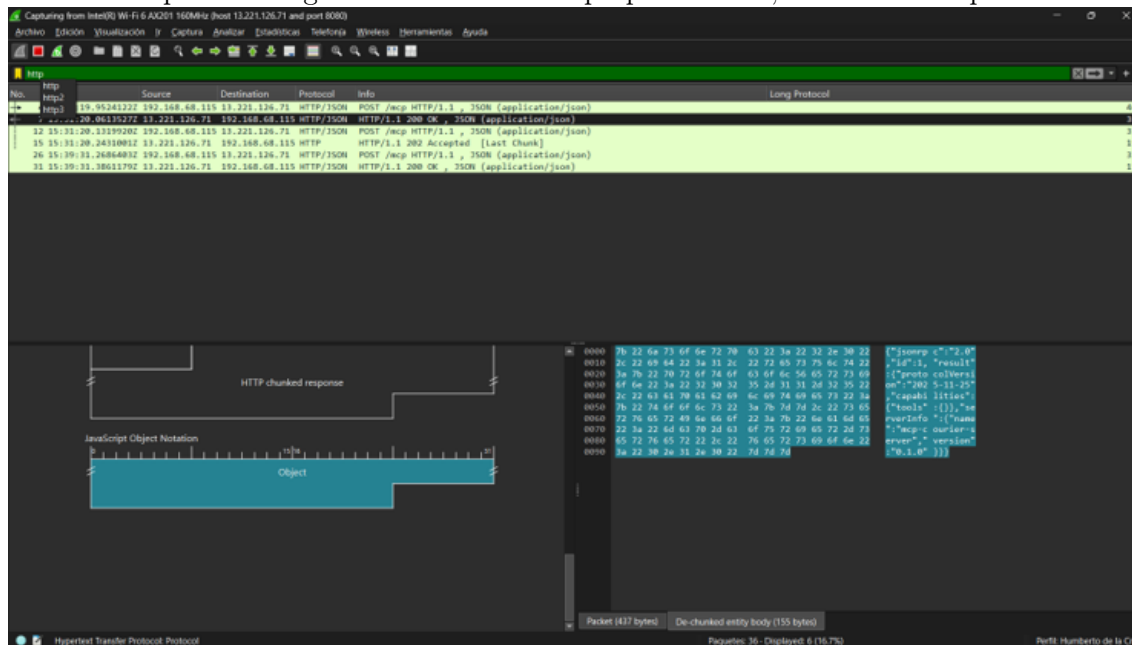
2.1 Configuración de la captura

Filtro de captura: host 13.221.126.71 and port 8080

Filtro de visualización: http

El servidor se expuso deliberadamente sobre HTTP en texto plano para permitir inspeccionar los mensajes JSON-RPC directamente. En un despliegue productivo se usaría TLS, y el contenido observado en Wireshark se limitaría a registros **Application Data** cifrados.

Los servidores locales (Filesystem y Git) no aparecen en esta captura: se comunican por **stdio**, es decir, *pipes* del sistema operativo que nunca atraviesan la interfaz de red. Su evidencia de funcionamiento corresponde al log de interacciones del propio chatbot, no a tráfico capturable.



2.2 Clasificación de los mensajes JSON-RPC

2.2.1 Mensajes de sincronización

Corresponden al establecimiento de la sesión MCP, antes de cualquier invocación de herramientas.

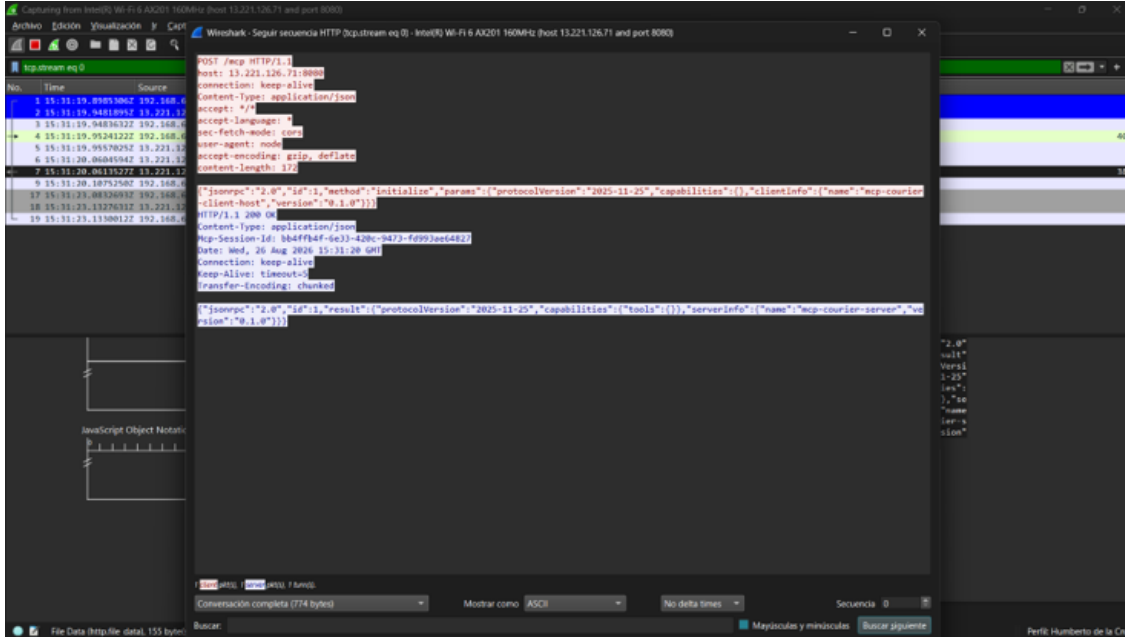
Petición initialize:

```
{"jsonrpc": "2.0", "id": 1, "method": "initialize",  
  "params": {"protocolVersion": "2025-11-25", "capabilities": {},  
    "clientInfo": {"name": "mcp-courier-client-host", "version": "0.1.0"}}}
```

Respuesta del servidor:

```
{"jsonrpc": "2.0", "id": 1,  
  "result": {"protocolVersion": "2025-11-25", "capabilities": {"tools": {}},  
    "serverInfo": {"name": "mcp-courier-server", "version": "0.1.0"}}}
```

Ambos extremos acuerdan la versión de protocolo 2025-11-25; el servidor declara en **capabilities** que expone herramientas. La respuesta incluye el encabezado **Mcp-Session-Id**, mecanismo mediante el cual MCP mantiene estado sobre un transporte (HTTP) que por diseño no lo conserva.



La notificación **notifications/initialized** carece de campo **id** — lo que la define como notificación en JSON-RPC 2.0, sin respuesta esperada — y el servidor la responde con HTTP 202 Accepted sin cuerpo, a diferencia del 200 OK de las peticiones normales. Ese contraste (200 vs. 202) distingue visualmente, en la captura, las peticiones de las notificaciones.

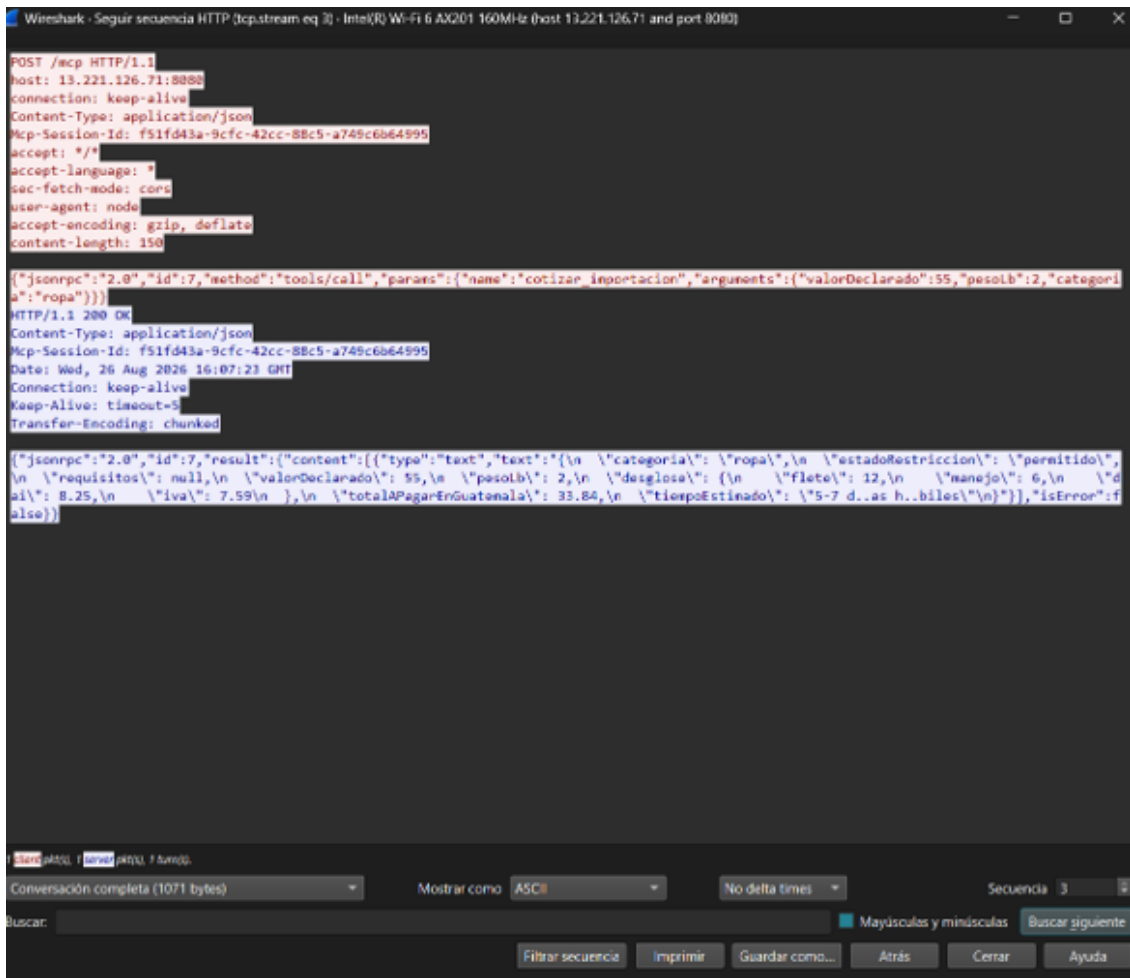
La petición **tools/list** (46 bytes) recibe como respuesta el catálogo completo de las seis herramientas con su **inputSchema** (3388 bytes); por su tamaño, esa respuesta viaja con **Transfer-Encoding: chunked** y se fragmenta en varios segmentos TCP.

2.2.2 Mensajes de solicitud

Cada invocación de herramienta es un método **tools/call** con su propio **id**. Ejemplo, a partir de la consulta del usuario *“Compré unos zapatos que pesan 2 libras y me costaron 55 dólares, ¿cuánto me sale traerlos a Guate?”*:

```
{
  "jsonrpc": "2.0",
  "id": 7,
  "method": "tools/call",
  "params": {
    "name": "cotizar_importacion",
    "arguments": {
      "valorDeclarado": 55,
      "pesoLb": 2,
      "categoria": "ropa"
    }
  }
}
```

El modelo construyó los argumentos a partir del lenguaje natural del usuario y del **inputSchema** obtenido previamente en **tools/list**; los nombres de los parámetros nunca fueron mencionados por el usuario.



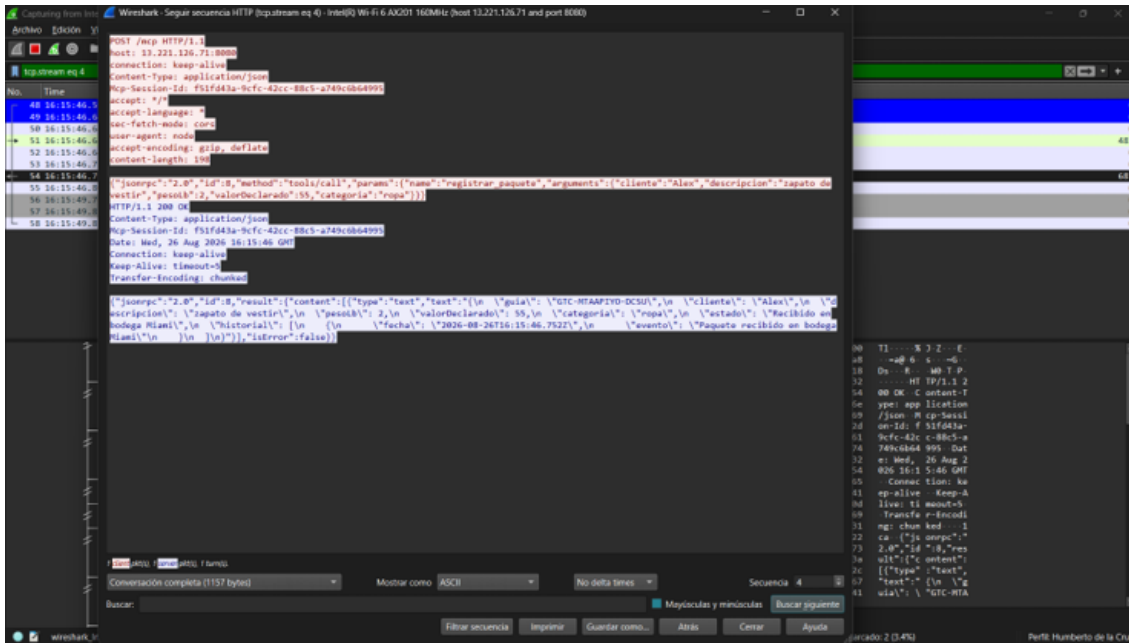
```
Wireshark · Seguir secuencia HTTP (tcp.stream eq 3) · Intel(R) Wi-Fi 6 AX201 160MHz (host 13.221.126.71 and port 8080)

POST /mcp HTTP/1.1
host: 13.221.126.71:8080
connection: keep-alive
Content-Type: application/json
Mcp-Session-Id: f51fd43a-9cfc-42cc-88c5-a749c6b64995
accept: */*
accept-language: *
sec-fetch-mode: cors
user-agent: node
accept-encoding: gzip, deflate
content-length: 150

{"jsonrpc":"2.0","id":7,"method":"tools/call","params":{"name":"cotizar_importacion","arguments":{"valorDeclarado":55,"pesolb":2,"categoria":"ropa"}}}
HTTP/1.1 200 OK
Content-Type: application/json
Mcp-Session-Id: f51fd43a-9cfc-42cc-88c5-a749c6b64995
Date: Wed, 26 Aug 2026 16:07:23 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Transfer-Encoding: chunked

{"jsonrpc":"2.0","id":7,"result":{"content":[{"type":"text","text":{"categoria":"ropa","estadoRestriccion":"permitido","requisitos":null,"valorDeclarado":55,"pesolb":2,"desglosa":{"flete":12,"manejo":6,"dai":8.25,"iva":7.59},"totalAPagarEnGuatemala":33.84,"tiempoEstimado":"5-7 d..as h..biles"},"isError":false}]}
```

En una llamada posterior a `registrar_paquete`, los valores de peso, valor y categoría se reutilizan de la consulta anterior sin que el usuario los repita — evidencia del mantenimiento de contexto de sesión: el servidor MCP no conserva memoria alguna entre llamadas; es el LLM quien retiene el historial que el anfitrión le reenvía en cada petición a la API.



2.2.3 Mensajes de respuesta

Toda respuesta conserva el id de la petición que la originó, lo que permite correlacionarlas incluso viajando sobre la misma conexión TCP.

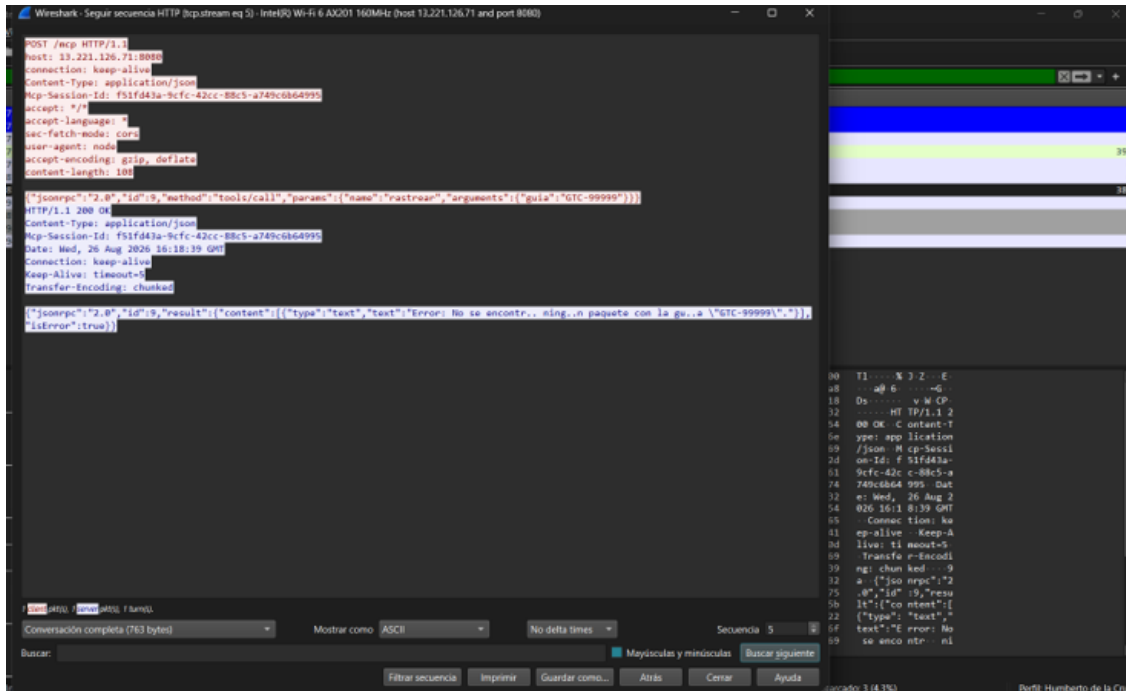
Es necesario distinguir dos mecanismos de error, observados ambos en la captura:

	Error de protocolo	Error de herramienta
Formato	<code>{.error":{"code":-32601,...}}{`</code>	<code>esult":{"...","isError":true}}</code>
Origen	La petición JSON-RPC es inválida	La herramienta se ejecutó y reportó un problema
Código HTTP	200	200
Destinatario	El cliente lo maneja	El LLM lo interpreta y lo explica al usuario

El caso capturado (consulta de una guía inexistente, id: 9) corresponde al segundo tipo: la petición era válida en todos sus aspectos — el método existe y el parámetro cumple el esquema — pero la lógica de negocio no encontró el registro:

```
{
  "jsonrpc": "2.0",
  "id": 9,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Error: No se encontro ningun paquete con la guia \"GTC-99999\".",
        "isError": true
      }
    ]
  }
}
```

MCP señala este caso con `isError` dentro de un resultado *exitoso* a nivel de protocolo, permitiendo que el modelo comunique la situación en lenguaje natural en vez de que el cliente deba interpretar un código de error genérico.



2.3 Análisis por capas

Capa de enlace (Ethernet II)

Direcciones MAC de origen y destino correspondientes a la interfaz Wi-Fi del equipo y al *gateway* de la red local. El campo Type indica IPv4 (0x0800). Las MAC cambian en cada salto de la ruta; no identifican al servidor remoto.

Capa de red (IPv4)

Origen 192.168.68.115, destino 13.221.126.71. La dirección de origen es privada, por lo que existe traducción NAT en el router doméstico: la instancia EC2 observa como origen la dirección pública de la red, la misma autorizada en el *security group*. El campo Protocol indica TCP (6) y el TTL decrece en cada salto del recorrido.

Capa de transporte (TCP)

Puerto de origen efímero, puerto de destino 8080. La conexión se establece mediante *three-way handshake* (SYN, SYN, ACK, ACK) y se mantiene con **Connection: keep-alive** (Keep-Alive: timeout=5): peticiones consecutivas reutilizan el mismo flujo TCP, evitando repetir el *handshake*, hasta que transcurren más de cinco segundos de inactividad. Respuestas extensas (p. ej. `tools/list`) se distribuyen en varios segmentos que TCP reordena y entrega completos a la capa superior.

Capa de aplicación (HTTP/1.1 + JSON-RPC 2.0)

Cada mensaje MCP viaja como cuerpo de un POST `/mcp` con **Content-Type: application/json**. JSON-RPC se ubica por encima de HTTP: define la estructura del mensaje (`jsonrpc`, `id`, `method`, `params`, `result`, `error`), mientras que HTTP actúa únicamente como transporte. Esta separación es la que permite que el mismo conjunto de mensajes funcione, sin modificación alguna, sobre `stdio` en el modo de ejecución local.

2.4 Resumen de la captura

Stream	Contenido	Clasificación
0	<code>initialize</code> + respuesta	Sincronización
0	<code>notifications/initialized</code> → 202	Sincronización (notificación)
2	<code>tools/list</code> + catálogo	Sincronización
3	<code>cotizar_importacion</code>	Solicitud + respuesta
4	<code>registrar_paquete</code>	Solicitud + respuesta
5	<code>rastrear</code> (guía inexistente)	Solicitud + respuesta de error

2.5 Observaciones

El servidor MCP no participa en la decisión de qué herramienta invocar: recibe mensajes JSON-RPC, ejecuta la operación solicitada y responde; desconoce por completo que existe un modelo de lenguaje del otro lado. La coordinación entre herramientas — incluido el uso de datos obtenidos en una consulta previa — ocurre en el anfitrión y en el LLM, nunca en el servidor.

La captura confirma que MCP es agnóstico respecto al transporte: los mensajes observados sobre HTTP son idénticos, en estructura, a los que el mismo servidor intercambiaría sobre `stdio` en su modo de ejecución local; únicamente cambia el canal por el cual viajan.

3 Conclusiones y comentarios sobre el proyecto

3.1 Dificultades encontradas y cómo se resolvieron

- **El servidor oficial de Git no tiene herramienta de inicialización.** Se revisó el código fuente instalado de `mcp-server-git` en cuatro versiones distintas (desde la 0.2.0 hasta la más reciente) y ninguna expone un `git_init`: todas sus herramientas asumen que el repositorio ya existe y fallan si no. Se resolvió inicializando el repositorio de trabajo una sola vez, fuera del chatbot, y documentando la limitación.
- **Incompatibilidad de versión en `mcp-server-git`.** La versión más reciente del servidor falla contra la versión más reciente del SDK `mcp` de Python (`AttributeError: 'Server' object has no attribute 'list_tools'`). Se resolvió fijando la versión del SDK con `uvx -with "mcp<2.0"mcp-server-git`.
- **Particularidades de Windows al lanzar procesos hijos.** Node no puede ejecutar directamente un `shim.cmd` de Windows (`npx.cmd`) sin pasar por una `shell`, pero habilitar `shell:true` de forma genérica hizo que `cmd.exe` interpretara el carácter `<` de `"mcp<2.0"` como un operador de redirección de archivo, rompiendo el arranque del servidor de Git. Se resolvió construyendo la línea de comando manualmente y con comillas propias para el caso de `npx`, y ejecutando `uvx` de forma directa (sin `shell`) para evitar por completo la interpretación de `cmd.exe`.
- **Inconsistencia de versión de Node.js entre terminales.** El transporte HTTP propio usa `fetch` nativo (disponible desde Node 18). Al ejecutar el proyecto desde una terminal que resolvía Node 16, el cliente fallaba silenciosamente con `fetch is not defined`, mientras que en otra terminal (con una versión distinta de Node en el `PATH`) funcionaba sin problema. Se resolvió estandarizando la versión de Node usada para ejecutar el proyecto.
- **Conectividad hacia el EC2.** Un intento de conexión SSH falló por *timeout*; la causa fue una regla del *security group* configurada como “My IP” que había quedado desactualizada tras un cambio de red. Relacionado con esto, se confirmó que la IP pública de la instancia cambia cada vez que se detiene y se vuelve a iniciar (a menos que se asigne una IP elástica), lo que obliga a reconfigurar la URL del servidor remoto en el cliente después de cada reinicio.
- **Integración con Claude Desktop como conector remoto.** Se intentó registrar el servidor remoto como conector MCP en Claude Desktop. Una primera configuración fue rechazada por faltar el campo `"type": "http"`; corregido ese campo, la aplicación lo siguió rechazando sin un mensaje de error específico en sus registros. Se investigó hasta revisar el binario de la aplicación sin encontrar la causa exacta, y se documentó como limitación abierta: no todo intento de integración se resuelve dentro del tiempo de un proyecto, y el requisito funcional real (que el propio chatbot use el servidor remoto igual que el local) ya estaba demostrado por otra vía.

3.2 Lecciones aprendidas

Implementar el protocolo a mano — en vez de usar el SDK oficial — hizo tangibles conceptos que normalmente quedan ocultos: la correlación de peticiones y respuestas mediante `id`, la distinción formal entre una petición y una notificación en JSON-RPC 2.0, los códigos de error estándar, y el manejo de sesión sobre un transporte (HTTP) que por diseño no conserva estado.

El diseño de MCP separa con claridad el protocolo (JSON-RPC, definido en términos de mensajes) del transporte (`stdio` o HTTP, que solo se encarga de llevar esos mensajes de un punto a otro).

Esa separación se demostró en la práctica: el mismo servidor, con la misma lógica de herramientas, funciona sin ningún cambio de código tanto en local por `stdio` como remoto por HTTP; únicamente cambia el módulo de transporte que se usa para hablar con él.

También quedó claro que la inteligencia del sistema — decidir qué herramienta usar, en qué orden, con qué argumentos, y cómo encadenar el resultado de una hacia la siguiente — vive enteramente en el anfitrión y en el modelo de lenguaje. Los servidores MCP, incluido el propio, son deliberadamente simples: ejecutan lo que se les pide y nada más.

3.3 Comentario general

El enunciado del proyecto menciona que “miles de desarrolladores y empresas han publicado sus servidores MCP”; este proyecto mostró en la práctica que esos servidores, incluidos los oficiales de Anthropic, no siempre están completos ni son perfectamente consistentes entre versiones — el caso del `git_init` ausente es el ejemplo más claro. Integrar varios componentes de distintos orígenes (un servidor propio, dos oficiales, y un proveedor de LLM externo) en un mismo anfitrión resultó ser, en la práctica, donde se concentró la mayor parte del esfuerzo de ingeniería del proyecto, más que en cualquier servidor individual.